Loop Introspection Solidification Plan

Review of the latest phased overhaul update and implementation plan for multi-target loop, gas, bytecode-size, and content-addressed analysis

Prepared for the implementor agent - 2026-05-24

> **Architectural north star: loop introspection must be a view over phase-owned identity and validated trace facts. It must not become a second compiler model. The same loop analysis substrate should serve Sean's Fibonacci investigation, gas attribution, bytecode-size attribution, content-addressed regression/fuzzing, LSP/CLI, and future multi-target backends.**

Review basis: static review of the current origin-overhaul-phased branch as visible on GitHub. I did not run cargo, trybuild, or the CLI inside this sandbox. Treat build/test status as unverified by me unless confirmed by your CI or local runs.

Contents

- 1. Executive verdict
- 2. Current branch review: what is now real, what is still a gap
- 3. Sean's Fibonacci case: evidence map and loop introspection gaps
- 4. Target loop-introspection architecture
- 5. Sonatina and multi-target backend requirements
- 6. Gas and bytecode-size attribution
- 7. Content-addressed loop views and granular fuzzing
- 8. Concrete phased implementation plan
- 9. CLI/LSP experience targets
- Appendices: fact schemas, query semantics, risks, future work, source review anchors

1. Executive verdict

The latest branch is in a much better architectural basin than the original broad spike. It has the right separation between developer trace fixtures, validated compiler trace JSONL, trace-query rendering, MIR-owned trace emission, and codegen-owned EVM bytecode/gas facts. The branch also has a serious content-addressing substrate in progress through shape-address and bytecode shape adapters.

The core gap is now narrow and important: loop contents are not yet a trustworthy compiler-derived object. The branch can show a fixture-backed Fibonacci loop and can emit real bytecode instruction/gas facts, but real trace loop-cost currently cannot prove which instructions are inside the loop, which are header/body/latch, which are per-iteration, which are due to MIR storage, and which are target legalization. That is exactly the next push to prioritize.

Decision	Recommendation
Merge posture	Do not call loop introspection complete yet. Treat the branch as a clean foundation plus a fixture UX and real trace substrate.
Next priority	Make loop regions first-class trace/query objects: blocks, CFG edges, instruction membership, loop structure, and derivation confidence.
Sean/Fibonacci target	Move from fixture claims about 13 instructions, 4 zexts, and b stack traffic to facts emitted or derived from MIR/Sonatina/backend.
Multi-target direction	Define target-neutral control-region and instruction-cost abstractions; let each backend add target-specific categories/cost models.
SSOT guardrail	Loop reports, gas reports, bytecode-size reports, and shape hashes are derived views. OriginExportKey remains the entity identity.

2. Current branch review: what is now real, what is still a gap

2.1 What is strong

- The README now makes the prototype boundary explicit: `fe dev trace-fixture` is fixture-backed, while `fe dev trace` reads validated trace JSONL and reports data source metadata. It also lists the current real trace commands and calls out loop membership and zext causality as gaps.
- `fe dev trace emit` is a real compiler trace path for standalone files. It resolves the target, builds a runtime package, emits MIR facts, compiles bytecode, emits bytecode instruction facts, validates the fact bundle, and writes JSONL.
- `fe dev trace query` now includes loop-cost, explain-local, gas-breakdown, explain-pc, gas-by-source, dynamic-gas-by-source, gas-to-source, optimized-code-honesty, and variables-at-pc command surfaces. This is the right command vocabulary for agentic probing, even though several reports depend on facts that are not yet emitted.
- `trace-facts` is still small and disciplined. It currently exports a tight set of fact variants: origin nodes/edges, compiler events, storage, instructions, instruction categories, loop membership, inline context, opcode, and gas cost. This is much safer than the old relation-system sprawl.
- MIR emits runtime local, statement, and terminator origin nodes plus storage facts. That is the right authority boundary: MIR owns runtime identity and MIR storage shape; codegen/backend owns final instructions and backend storage.

- Codegen emits actual EVM bytecode instruction facts, opcode facts, categories, and conservative static gas from emitted runtime bytecode. This is a real basis for bytecode-size and gas reporting.
- Content addressing has been reintroduced in a phase-appropriate way: `shape-address` supports dimensions, owner-bound versus anonymous views, graph hashing, and cycle policies; codegen has a bytecode shape adapter that separates opcode structure from PUSH immediates.

2.2 Current limitations to treat as design input

- Loop membership is the central missing piece. Current real traces can summarize bytecode, but if no LoopMembership facts exist, loop-cost correctly reports that loop cost is unavailable and lists required next facts.
- MIR storage reasons appear conservative in the currently visible branch. Runtime locals get StorageFact entries, but reasons are Unknown in the reviewed code. That is acceptable for a first trace, but Sean's `b` question needs real reasons such as MutableLocalLowering, AddressTaken, or ValueKeptSsa.
- Source-local display names are not yet present in the visible MIR trace module. That means `explain-local --local b` can be excellent in the fixture and weak in real compiler traces unless DisplayName or Variable facts are added.
- MIR-to-codegen and MIR-to-bytecode origin edges are not present in the real trace path. Without those edges, gas/bytecode-size reports can count bytecode but cannot reliably attribute it to source/MIR causes.
- Integer zero-extension causality is present in the fixture through CompilerEventKind::InsertIntegerZeroExtend and IntegerLegalizationFor edges, but the README correctly says no `zext-report` should be exposed until compiler phases emit those events and value-property facts.
- The shape CLI module exists and is useful, but current top-level CLI wiring was not visible in main.rs. Treat shape CLI exposure as something to verify locally or wire explicitly in the next push.
- The trace snapshot hash uses a simple debug hash in the reviewed file. That is fine for non-security trace identity, but content-addressed artifacts, package-like caching, and crypto-adjacent attestations should use BLAKE3-256 or SHA-256 with explicit domain separation.

2.3 Evidence map from the reviewed branch

Area	Observed current state	Implication
README trace status	Fixture-backed Fibonacci UX is explicit; real trace emits MIR facts and EVM bytecode instruction facts; loop membership and zext causality remain gaps.	Good honesty. Keep this language, but sync it after every push.
CLI commands	Dev trace has emit/validate/query/live plus loop-cost and explain-local shortcuts; query subcommands include gas and PC/source attribution surfaces.	Command vocabulary is ahead of fact emission. This is acceptable if reports clearly state missing facts.
Real trace emit	Standalone file only; emits MIR facts, compiles via Sonatina, emits EVM bytecode instruction facts, validates and writes JSONL.	Next: ingot targets, source names, cross-phase edges, loop facts.
Trace facts	Small fact vocabulary includes loop membership and integer legalization concepts but not broader debug/type/source/span facts in the visible branch.	Good minimality. Add only loop/debug facts that are needed, with validator coverage.
MIR trace	MIR-owned runtime nodes and StorageFact entries exist; backend registers/slots are explicitly not MIR-	Correct authority boundary; strengthen storage reasons/names.

	owned.	
Codegen trace	Bytecode instruction facts, opcode facts, static gas facts, and instruction categories are emitted from real EVM bytecode.	Good foundation for gas and bytecode-size attribution.
Trace rendering	Loop-cost says unavailable when loop facts are missing and lists the next facts needed.	This is the right trust posture. Preserve it.
Fixture	Fibonacci fixture encodes 13 instructions, 4 zexts, stack slot sp+24 for b, and zext compiler events.	Fixture is a useful target UX, not proof from the compiler.
Shape addressing	Bytecode shape graph separates opcode structure from constants and has cycle-policy support in shape-address.	Strong base for content-addressed regression and fuzzing; integrate with trace later as derived shape facts.

3. Sean's Fibonacci case: evidence map and loop introspection gaps

Sean's concrete question is about loop contents: why Fe's generated Fibonacci loop appears to contain repeated zero-extension, stack load/store traffic for `b`, and more loop-carried movement than the reference output. The right target is a report that does not merely list bytes or instructions, but explains which loop-region instructions are direct source intent, which are necessary lowering, which are target legalization, which are storage/spill traffic, and which are unmapped or ambiguous.

Claim we want to answer	**Current confidence**	**Needed for compiler-derived confidence**
The loop has 13 static instructions and 4 zexts.	Fixture-backed for the RISC-V-style demo; real EVM trace can count bytecode but cannot yet isolate this source loop.	Instruction-to-block facts, CFG edges, LoopFact/LoopMembership from compiler analysis, target-specific instruction categories.
`b` is stack-resident at sp+24.	Fixture-backed. Real MIR can say locals are memory-like or SSA-like, but backend final stack/register location is not emitted in the visible branch.	Backend storage allocation facts: stack slots, registers, EVM stack/memory/storage locations, and LoadOf/StoreOf edges.
`b` became memory-like in MIR.	Partially supported: MIR emits StorageFact with MemoryPlace/SsaValue based on runtime local roots, but storage reason/name fidelity must be strengthened.	MIR storage reason, source-local display/Variable facts, and origin edges from source local to runtime local.
`n` is loop-invariant and its zext is hoistable.	Fixture narrative only unless def/use/mutation/loop facts exist.	ValueDefFact, ValueUseFact, ValueMutationFact, loop membership, dominance or natural-loop analysis, and ValuePropertyFact for loop invariance.
`i` zext may be redundant after addiw.	Fixture narrative only.	Target value-property propagation: known-u32, known-zero-extended, upper-bits-known, plus backend InsertIntegerZeroExtend events.
Inlining made the loop stackier.	Not proven yet in real traces.	InlineContextFact emitted for real, plus per-instance loop and storage facts for standalone versus inlined function bodies.
The EVM bytecode/gas impact of the loop is attributable to source.	Bytecode/gas facts exist, but source attribution edges are missing.	Bytecode PC -> Sonatina/MIR/HIR/source edges plus

		attribution policy.

3.1 The loop report Sean ultimately needs

```
fe dev trace query loop-contents --from target/fib.trace.jsonl --function Fib.recv --loop 0

Loop: Fib.recv Compute handler / while i < n
Derivation: compiler natural-loop analysis, cfg_hash=...
Target: evm/sonatina/cancun

Per-iteration static region:
  instructions: 13
  byte size: 31 bytes
  static gas: 68 conservative opcode gas
  stack/local movement: 2 instructions
  integer normalizations: 4 instructions
  direct user arithmetic: 2 instructions
  branches/jumps: 2 instructions

Findings:
  b: memory-like from MIR MutableLocalLowering; backend location = stack slot / EVM stack-
memory slot
  n: loop-invariant; normalization occurs inside loop; candidate = hoist/canonicalize
  i: normalized before compare; known-u32 fact after increment missing

Confidence:
  loop membership: compiler_cfg
  instruction categories: backend_emission_reason
  zext causes: backend_event
  gas: conservative_static_opcode
  source attribution: origin_edge_chain
```

4. Target loop-introspection architecture

The current LoopMembershipFact is useful but too thin for solid multi-target analysis. We should keep it as a derived membership view, but add the underlying region model that makes membership auditable. The structure below keeps identity owned by phase crates and makes loop reports downstream, validated views.

4.1 Core concepts

Concept	Owner	Purpose
Block identity	MIR/Sonatina/backend phase crates	Names a basic block in the phase that owns it. Required for CFG and instruction membership.
CFG edge	Owning phase or derived loop-analysis pass	Represents control-flow successor/predecessor relationships with edge kind.
Loop/region identity	Derived analysis over phase-owned CFG	Names a natural loop or target region without pretending to be a compiler IR node.
Block role in loop	Derived analysis	Header, body, latch, preheader, exit. Distinguishes setup from per-iteration work.
Instruction membership	Backend/codegen or block-layout emitter	Maps final instruction/PC to block and then to loop.
Value def/use/mutation	Owning phase	Lets the query layer derive loop

		invariance and repeated work.
Compiler event	Phase that introduced the work	Explains zext/signext/spill/reload/register move/inlining events.
Cost facts	Target backend or cost model	Gas, bytes, estimated cycles, or target-specific cost metrics.

4.2 Proposed fact additions

These facts should be added only as typed trace-facts with validator coverage. Relation/Datalog rows must be generated or derived from them, never hand-mirrored.

```
TraceFact::Block(BlockFact)
TraceFact::CfgEdge(CfgEdgeFact)
TraceFact::Loop(LoopFact)
TraceFact::LoopBlock(LoopBlockFact)
TraceFact::InstructionBlock(InstructionBlockFact)
TraceFact::InstructionExtent(InstructionExtentFact)
TraceFact::ValueDef(ValueDefFact)
TraceFact::ValueUse(ValueUseFact)
TraceFact::ValueProperty(ValuePropertyFact)
TraceFact::DisplayName(DisplayNameFact)        // or VariableFact if broader debug model lands
TraceFact::SourceSpan(SourceSpanFact)          // optional for source attribution
TraceFact::ShapeNodeHash/ShapeGraphHash        // derived content-addressed view, not identity
```

Fact	Minimum fields	Validator requirements
BlockFact	block_key, function_key, phase, ordinal/name	block and function are origin nodes; phase matches kind; no duplicate block ordinal per function.
CfgEdgeFact	function_key, from_block, to_block, edge_kind, condition_origin?	endpoints exist; function matches both blocks; edge kind non-empty enum.
LoopFact	loop_key, function_key, phase, header_block, derivation, confidence	header exists; function exists; derivation has non-empty cfg_hash if NaturalLoopAnalysis.
LoopBlockFact	loop_key, block_key, role	loop/block exist; exactly one header; latches/exits may be plural.
InstructionBlockFact	instruction_key, block_key, phase/target	instruction/block exist; instruction has one primary block.
InstructionExtentFact	instruction_key, code_object, address/pc_start, pc_end, byte_len	valid non-empty range; no duplicate exact instruction extents.
ValueDef/ValueUse	value_or_local, instruction_or_stmt, block, phase	subjects and sites exist; phase consistent.
ValuePropertyFact	subject, phase, property, evidence	property enum; evidence references known fact/event where possible.

4.3 Loop confidence levels

Level	Meaning	Allowed report language
fixture	Loop membership hard-coded for a demo.	UX demo only; not compiler evidence.
posthoc_disassembly	Loop inferred from final code patterns without phase CFG.	Heuristic; use words like inferred/candidate.
backend_block_mapping	Backend emitted block-to-instruction ranges.	Trustworthy final-code region, but may not map to source loop without origin edges.
sonatina_cfg	Sonatina block graph and instruction membership emitted.	Good compiler-derived loop region at backend IR level.

mir_cfg	MIR runtime CFG emitted and linked to source/HIR.	Good for source-level loop and storage reasoning.
cross_phase_witness	MIR/Sonatina/backend loops linked through origin edges and transformations.	Strongest ordinary confidence; suitable for Sean-style diagnosis.

5. Sonatina and multi-target backend requirements

Sean is working on the multi-target backend, so the loop-introspection design must not assume EVM-only bytecode or RISC-V-style assembly. The shared model should describe code objects, blocks, CFG edges, instructions, origins, costs, and storage locations in target-neutral terms, while each backend supplies target-specific instruction categories and cost models.

5.1 Required Sonatina-side instrumentation

1. Emit Sonatina function, block, and instruction origin keys. They should be owned by the codegen/Sonatina crate, not common.
2. Emit instruction-to-block membership for pre-opt and post-opt Sonatina snapshots.
3. Emit CFG edges for each Sonatina function, including branch/conditional/fallthrough/call/return edge kinds as applicable.
4. Preserve MIR/runtime origins through Sonatina lowering. RuntimeStmt/RuntimeTerminator origins should link to the Sonatina instructions they produce via LoweredFrom or EmittedFrom-style edges.
5. Emit optimization snapshot joins honestly: same instruction ID after optimization is snapshot identity, not proof of transformation preservation. Created, elided, rewritten, split, and merged cases should be explicit where known and Unknown/Ambiguous where not.
6. Emit compiler events for target legalization: integer zero/sign extension, register moves, phi/block-parameter materialization, stack-slot creation, spills, reloads, ABI/setup, and backend-prepared code.
7. Emit final instruction/PC extent facts from the backend and link them to Sonatina post-opt instructions. For EVM, PC and byte length are enough; for native-like targets, address/range or asm index may be used until object emission is richer.

5.2 Multi-target adapter contract

```
trait TargetTraceEmitter {
    fn target_id(&self) -> TargetId;              // evm/cancun, riscv64, wasm32, etc.
    fn emit_code_objects(&self) -> Vec<TraceFact>;
    fn emit_blocks_and_cfg(&self) -> Vec<TraceFact>;
    fn emit_instructions(&self) -> Vec<TraceFact>;
    fn emit_instruction_extents(&self) -> Vec<TraceFact>;
    fn emit_instruction_categories(&self) -> Vec<TraceFact>;
    fn emit_cost_facts(&self) -> Vec<TraceFact>;  // gas, bytes, cycles, or unknown
    fn emit_storage_locations(&self) -> Vec<TraceFact>;
    fn emit_origin_edges(&self) -> Vec<TraceFact>;
}
```

The exact Rust interface can differ, but the boundary should be this clear: target backends emit target-owned final-code facts; they do not reconstruct HIR/MIR identity. Cross-phase edges are emitted only when the backend has a phase-owned mapping from lowering/codegen.

Target	Loop contents require	Cost facts require	Storage facts require
EVM	PC-to-basic-block or jumpdest region	Static opcode gas, dynamic gas from EVM trace, byte	EVM stack depth, memory ranges, storage slots,

	membership; source loop may require MIR/Sonatina mapping.	length per instruction.	calldata ranges.
RISC-V/native-like	Asm/object instruction-to-block ranges and branch backedges.	Instruction count, byte size, target cycles/latency if model exists.	Virtual/physical registers, stack slots, frame offsets, spills/reloads.
Wasm/future targets	Block/structured-control regions or CFG lowering to blocks.	Encoded byte length, estimated instruction cost, runtime trace if available.	Locals, stack values, linear memory ranges.

6. Gas and bytecode-size attribution

Gas and bytecode-size questions are adjacent to loop contents: the same instruction membership, source attribution, and cost facts should support all three. Do not build a separate gas-specific source map. Gas and byte-size reports should be derived from the same target-neutral instruction and origin graph.

6.1 Static gas

- The current EVM bytecode trace already emits conservative static opcode gas from emitted bytecode. Preserve the confidence label: this is an opcode-table estimate, not full transaction gas.
- Dynamic costs such as memory expansion, storage warm/cold access, call costs, copy sizes, and keccak input lengths require runtime or richer static facts. Represent them as DynamicGasKind or CostComponent facts, not as a fake static scalar.
- Attribution must name its policy: exclusive-primary, inclusive, synthetic-overhead, call-inclusive, call-exclusive. Default LSP/inlay hints should use conservative exclusive-primary plus separate synthetic overhead where possible.

6.2 Bytecode size

- Add instruction extent facts so bytecode size can be derived directly from pc_start/pc_end or byte_len. For EVM PUSH instructions, the immediate bytes must count toward byte size and constants dimension hashing.
- Group bytecode size by source and by compiler overhead class: direct source operation, loop control, ABI/setup, storage movement, integer legalization, optimizer-created, backend-prepared, unmapped.
- For multi-target, keep byte size separate from gas/cycles. A bytecode-size improvement may harm runtime gas and vice versa. Reports should expose vectors, not one overloaded cost number.

6.3 Target reports

```
fe dev trace query gas-breakdown --from target/fib.trace.jsonl --schedule cancun
fe dev trace query gas-by-source --from target/fib.trace.jsonl --schedule cancun --policy
exclusive-primary
fe dev trace query bytecode-size-by-source --from target/fib.trace.jsonl --policy synthetic-
overhead
fe dev trace query loop-cost --from target/fib.trace.jsonl --target evm --loop <loop-key>
```

7. Content-addressed loop views and granular fuzzing

Content addressing should become the regression/fuzzing backbone for loop introspection. A loop region should have dimensioned hashes at each phase: HIR loop region, MIR loop subgraph, Sonatina loop CFG/component, and final bytecode/target region. These hashes are derived facts, never compiler identity.

Dimension	Loop use case
structure	Detect control-flow/instruction-shape changes independent of names and constants. Useful for bucketing fuzz cases and detecting compiler-regression shape shifts.
constants	Detect literal/PUSH immediate changes without conflating them with structural shape. Useful for parameterized crypto circuits/contracts.
names	Detect display/name-only changes when source/IR structure is otherwise unchanged. Mostly diagnostic.
types	Detect type-driven lowering changes, integer-width normalization, and ABI layout differences.
trace-events	Detect changes in compiler-generated overhead: zext insertion, spills, synthetic checks, optimizer-created nodes.

7.1 Loop hash facts

```
Loop shape emission should produce derived facts like:

ShapePolicyFact { policy_id, dimensions, algorithm, cycle_policy }
ShapeNodeHashFact { node, phase, policy_id, local_hashes, tree_hashes }
ShapeComponentHashFact { component, members, phase, policy_id, hashes }  // SCC or region
ShapeGraphHashFact { graph_key, phase, policy_id, hashes }

These are derived content views keyed by OriginExportKey. They must not replace OriginExportKey
identity.
```

7.2 Fuzzing easy wins

- Bucket generated loop regions by structure hash and fuzz one representative per bucket before expanding to all duplicates.
- When a compiler change occurs, compare the smallest changed hash component to focus fuzzing on the affected loop/IR subgraph.
- Use trace-events hashes to fuzz cases where synthetic work changed, even if source/HIR structure did not.
- Use anonymous-shape mode to detect repeated structural patterns across functions/contracts; use identity-bound mode when debugging one specific source region.
- For cyclic graph-like IRs, use SCC condensation rather than rejecting useful loops. Preserve a policy knob so tree-like IRs can still reject cycles as a bug.

8. Concrete phased implementation plan

Phase 0 - align status and tests

- Synchronize README/status output with the actual current fact emission. If source-local display names or storage reasons are not emitted, say so.
- Ensure `fe dev trace status` distinguishes fixture, compiler-emitted, posthoc, and unavailable facts.
- Add a golden test that real `dev trace emit` followed by `loop-cost` says loop cost unavailable until LoopMembership exists.
- Add a golden test that fixture loop-cost still demonstrates the target UX and is clearly labeled fixture-backed.

Acceptance: Passes if all CLI outputs clearly state confidence/provenance and no report overclaims loop membership.

Phase 1 - target-neutral CFG and loop facts

- Add BlockFact, CfgEdgeFact, LoopFact, LoopBlockFact, InstructionBlockFact, and InstructionExtentFact to trace-facts.
- Extend TraceValidator for missing endpoints, duplicate block ordinals, invalid extents, bad cfg hashes, and loop role invariants.
- Keep existing LoopMembershipFact as a derived convenience row or make it generated from LoopBlock + InstructionBlock.

Acceptance: Passes if a synthetic test can validate a small CFG with one natural loop and reject malformed loop facts.

Phase 2 - MIR loop emission

- Emit MIR block facts and CFG edges for runtime functions.
- Run natural-loop analysis over MIR CFG and emit LoopFact/LoopBlock facts.
- Emit RuntimeStmt/Terminator -> block membership and instruction extent analogs for MIR sites.
- Emit source-local DisplayName/Variable facts and StorageReason values such as MutableLocalLowering or ValueKeptSsa.

Acceptance: Passes if real Fibonacci trace can identify the MIR loop and explain whether b is memory-like in MIR without fixture facts.

Phase 3 - Sonatina loop and origin bridge

- Emit Sonatina function/block/instruction facts pre-opt and post-opt.
- Emit Sonatina CFG and loop facts, plus instruction-to-block membership.
- Link Sonatina instructions to MIR statements/terminators where lowering has that information.
- Emit optimizer snapshot classifications without overclaiming transformation provenance.

Acceptance: Passes if a real trace can show MIR loop -> Sonatina loop relationship and distinguish pre/post opt region membership.

Phase 4 - backend/target final-code facts

- Emit final bytecode/asm instruction extent facts and block membership for EVM and any new multi-target backend Sean is working on.
- Emit target-specific cost facts: EVM static gas, byte size, and future cycle/latency estimates where available.
- Emit backend storage locations: EVM stack/memory/storage/calldata or native stack/register slots.
- Emit compiler events for zext/signext/spill/reload/register move when the backend actually performs them.

Acceptance: Passes if real loop-cost can report final-code contents for at least one target with honest categories and costs.

Phase 5 - source/gas/byte-size attribution

- Add or strengthen MIR/Sonatina/backend origin edges so bytecode PCs can reach source/HIR/MIR causes.
- Implement gas-by-source and bytecode-size-by-source on real traces with explicit attribution policy.
- Implement explain-pc showing instruction, cost, source candidates, confidence, and origin chain.

Acceptance: Passes if real Fibonacci/EVM trace can show gas and byte size for attributed source regions, while unmapped costs remain explicit.

Phase 6 - content-addressed loop views

- Emit shape hashes for loop regions at MIR, Sonatina, and bytecode/final-code levels.
- Expose shape diff/bucket commands through the real CLI if not already wired.

- Add loop-compare reports showing which phase and dimension changed between two trace bundles.

Acceptance: Passes if a compiler tweak can show source unchanged, MIR/shape unchanged or changed, codegen shape changed, and final loop-cost delta.

Phase 7 - LSP/live/browser integration

- Surface loop-cost and explain-local hovers/inlays from validated trace snapshots or live LSP endpoint data.
- Add code lenses: Explain loop, Explain local storage, Gas by source, Bytecode size by source, Open trace graph.
- Expose graph JSON using OriginExportKey and shape hashes, not ad hoc string IDs.

Acceptance: Passes if an editor can show a concise loop hint and a detailed hover backed by the same trace facts as the CLI.

9. CLI and LSP experience targets

The command surface should make it obvious what is real, what is fixture-backed, what is inferred, and what is missing. Sean should be able to use the current tools as a hypothesis generator now, and later as compiler evidence.

9.1 Current commands to verify locally

```
cargo run -p fe -- dev trace emit fib_demo.fe --out target/fib.trace.jsonl
cargo run -p fe -- dev trace validate --from target/fib.trace.jsonl
cargo run -p fe -- dev trace loop-cost --from target/fib.trace.jsonl
cargo run -p fe -- dev trace explain-local --from target/fib.trace.jsonl --local b

cargo run -p fe -- dev trace query gas-breakdown --from target/fib.trace.jsonl
cargo run -p fe -- dev trace query explain-pc --from target/fib.trace.jsonl --pc 0
cargo run -p fe -- dev trace query gas-by-source --from target/fib.trace.jsonl

cargo run -p fe -- dev trace-fixture loop-cost fib_demo.fe
cargo run -p fe -- dev trace-fixture explain-local fib_demo.fe --local b
```

9.2 Commands to add once loop facts land

```
fe dev trace query loop-contents --from target/fib.trace.jsonl --loop <loop-key>
fe dev trace query loop-cost --from target/fib.trace.jsonl --phase mir
fe dev trace query loop-cost --from target/fib.trace.jsonl --phase sonatina-post-opt
fe dev trace query loop-cost --from target/fib.trace.jsonl --target evm
fe dev trace query bytecode-size-by-source --from target/fib.trace.jsonl --policy synthetic-
overhead
fe dev trace query loop-shape-diff --left target/before.trace.jsonl --right
target/after.trace.jsonl --dimension structure
fe dev trace query loop-fuzz-buckets --from target/corpus/*.trace.jsonl --dimension structure
```

9.3 LSP display rules

UI surface	Show	Do not show
Inlay hint	Short loop summary: "13 instr/iter, 4 zext, 2 stack ops" or "gas ~N" with confidence if not high.	Full origin chains, long warnings, speculative paragraphs.
Hover on loop	Loop contents, source attribution, per-iteration summary, confidence, missing facts.	Unlabeled fixture or posthoc claims.
Hover on local	Storage history, loop accesses, earliest memory-like phase, likely compiler	Fake backend stack slots unless emitted by backend facts.

	area.	
Code lens	Explain generated code, Gas by source, Bytecode size, Open graph, Compare shape.	Always-on noisy lenses on every line.
Browser graph	OriginExportKey nodes, CFG blocks, loop region, instruction extents, cost facts, shape hashes.	Graph nodes identified only by names or line numbers.

Appendix A - Query semantics for solid loop introspection

```
Base facts:
  block(function, block, phase)
  cfg_edge(function, from_block, to_block, edge_kind)
  loop(loop_key, function, phase, header_block, derivation)
  loop_block(loop_key, block, role)
  instruction_block(instruction, block)
  instruction_extent(instruction, code_object, pc_start, pc_end, byte_len)
  origin_edge(from, to, label)
  storage(subject, phase, location, reason)
  compiler_event(event, phase, kind, inputs, outputs, reason)
  gas_cost(instruction, schedule, gas, confidence)

Derived relations:
  inst_in_loop(Inst, Loop) :- instruction_block(Inst, Block), loop_block(Loop, Block, _).
  loop_header_inst(Inst, Loop) :- instruction_block(Inst, Block), loop_block(Loop, Block,
header).
  origin_reaches(A, B) :- origin_edge(A, B, _).
  origin_reaches(A, C) :- origin_edge(A, B, _), origin_reaches(B, C).
  gas_for_loop(Loop, sum<Gas>) :- inst_in_loop(Inst, Loop), gas_cost(Inst, _, Gas, _).
  stack_traffic_for_local(Loop, Local, Inst) :- inst_in_loop(Inst, Loop), origin_edge(Inst,
Local, load_of_or_store_of).
  zext_for_local(Loop, Local, Inst) :- inst_in_loop(Inst, Loop), origin_edge(Inst, Local,
integer_legalization_for).
  loop_invariant_candidate(Local, Loop) :- used_in_loop(Local, Loop), not
mutated_in_loop(Local, Loop).
```

Appendix B - Risks, caveats, and workarounds

Risk	Why it matters	Mitigation
Loop facts become a second CFG	If loop facts are hand-authored separately from CFG, they can drift.	Derive loop facts from emitted CFG or carry cfg_hash and validation.
Source attribution double-counts gas/bytes	One instruction may have many source ancestors.	Require attribution policy in every report. Separate synthetic overhead from user-authored work.
Optimizations destroy one-to-one mapping	Inlining, unrolling, CSE, DCE, and merging make exact source mapping impossible.	Use classifications: direct, synthetic, optimizer-created, ambiguous, unmapped. Do not fake precision.
Static gas is mistaken for transaction gas	EVM gas is context-dependent.	Label static opcode gas as conservative; ingest runtime traces for measured gas.
Multi-target costs are incomparable	Gas, bytes, cycles, and stack pressure are different metrics.	Report cost vectors and target-specific models; avoid one universal cost scalar.
Trace volume grows too large	Full instruction/block/value facts can be big.	Support filtering by function/loop/target; use content-addressed snapshots and optional detail levels.
Hashing is overtrusted	Shape hashes detect structural equality under a policy, not semantic equivalence.	Every shape diff must say "not semantic equivalence"; version policies and algorithms.
Security-sensitive attestation uses debug hash	Fast debug hashes are not collision-resistant.	Use BLAKE3-256 or SHA-256 for content addressing/attestation with domain separation.

Appendix C - Forward-looking big-picture threads

C.1 Full-stack crypto and zk/EVM work

For Sean's full-stack zk/EVM demo, loop introspection is one instance of a broader high-assurance story: the compiler should make proof-relevant, gas-relevant, and safety-relevant lowering visible. This does not require claiming broad semantic preservation. It requires concrete structural witnesses and content-addressed artifacts.

- Verifier/check bottleneck witness: source check -> HIR condition -> MIR branch/dominator region -> bytecode control-flow region -> sensitive operation dominated by the check.
- Gas/security tradeoff reports: static/dynamic gas by source region, with synthetic overhead separated from user-authored proof/check logic.
- Bytecode-bloat reports: identify ABI/runtime setup, checks, proof verifier body, field arithmetic, memory/call data movement, and optimizer-created code.
- Content-addressed compiled artifacts: source hash, HIR/MIR/Sonatina/bytecode shape hashes, debug trace hash, and bytecode hash bundled as an audit artifact.
- Fuzzing: bucket structurally unique verifier loops/circuits and focus fuzzing on changed subgraphs after compiler optimization changes.

C.2 Bidirectional-transformation adjacency

The trace/content-addressed spine is not a bidirectional compiler, but it is the correct prerequisite. To support future MIR-level edits that can suggest HIR/syntax changes, forward passes must record the complement of lost information: grouping, source constructs, types, desugarings, synthetic reasons, and non-invertible merges. Loop introspection will exercise many of the same muscles: one-to-many lowering, many-to-one optimized instructions, synthetic events, and explicit unknowns.

C.3 Browser/Webscape-style graph views

A browser graph should be a view over TraceFacts and ShapeFacts, not a separate model. Nodes should be OriginExportKey values; edges should be origin/CFG/loop/cost/shape relations. A useful first graph for Sean would render the Fibonacci loop with four highlighted chains: zext of i, zext of n, load of b, store of b.

Appendix D - Source review anchors

Path / area	Observation used in this report
README.md / Developer Trace Prototype	Documents fixture-backed trace UX, real trace commands, and current gaps: loop membership, source-local labels, storage allocation, zext causality.
crates/fe/src/main.rs	Defines `fe dev trace`, fixture commands, query commands, live LSP query surface, and gas/source report commands.
crates/fe/src/trace/trace_emit.rs	Real trace emit path: standalone file -> MIR facts -> Sonatina bytecode -> bytecode instruction facts -> validation -> JSONL.
crates/trace-facts/src/fact.rs	Small fact model with origin, storage, instruction, loop membership, inline, opcode, gas, and compiler event concepts.
crates/trace-facts/src/validate.rs	Validator checks endpoints, duplicate instructions/sites, malformed strings, invalid storage/gas/opcode relationships.
crates/mir/src/trace.rs	MIR emits runtime local/statement/terminator nodes and

	storage facts; backend facts are intentionally out of MIR scope.
crates/codegen/src/trace.rs	Codegen emits actual EVM bytecode instruction, opcode, category, and conservative static gas facts.
crates/fe/src/trace/trace_render.rs	Loop-cost renderer explicitly says unavailable when loop membership is missing and lists required next facts.
crates/fe/src/trace/trace_fixture.rs	Fibonacci fixture encodes the 13-instruction demo, b stack slot, zexts, and compiler events for UX demonstration.
crates/shape-address/src/lib.rs	Shape-address supports dimensions, owner-bound/anonymous views, cycle policies, and graph hashing.
crates/codegen/src/shape.rs	Bytecode shape graph separates opcode structure from PUSH immediate constants.
crates/fe/src/shape_cli.rs	Shape emit/explain/diff/bucket module exists; verify top-level CLI wiring in local branch.